Análisis de complejidad del algoritmo

Problema de maximización de ganancias en acciones por División y Conquista

Definimos T(n) como el tiempo que tarda AnalizarIntervalo(precios, inicio, fin) en resolver un intervalo de n precios.

La función devuelve tres datos: (mínimo, máximo, ganancia).

La variable ganancia representa la mejor ganancia válida dentro del intervalo, respetando el orden compra → venta.

Queremos obtener una recurrencia para T(n) y resolverla.

1. Construcción de la recurrencia

Caso base

Si el intervalo tiene un solo elemento:
AnalizarIntervalo([x]) = (x, x, 0)

Costo del caso base:
$T(1) = \Theta(1)$

Paso recursivo

Para un intervalo de tamaño n, el algoritmo divide en dos mitades.

Resuelve recursivamente la mitad izquierda y la mitad derecha: $2T(n/2)$.

Luego combina los resultados usando solamente:

mínimo = min(min1, min2)

máximo = max(max1, max2)

gananciaCruce = max2 − min1

ganancia = max(ganancia1, ganancia2, gananciaCruce)

2. ¿Por qué el costo de combinación es Θ(1)?

La etapa de combinación no vuelve a recorrer el arreglo.

Cada rama ya devolvió la información necesaria: mínimo, máximo y mejor ganancia.

Por lo tanto, combinar exige solo una cantidad fija de operaciones:

- comparaciones
- una resta
- algunas asignaciones

Ese número de operaciones no depende de n.

Recurrencia resultante:
$$T(n) = 2T(n/2) + \Theta(1)$$

Costo de combinación = Θ(1)

3. Análisis mediante árbol de recurrencia

En el nivel 0 hay 1 llamada y cada una cuesta Θ(1) fuera de la recursión.

En el nivel 1 hay 2 llamadas.

En el nivel 2 hay 4 llamadas.

En general, en el nivel i hay 2^i llamadas.

Entonces, el costo total del nivel i es:

$$2^i \cdot \Theta(1) = \Theta(2^i)$$

La recursión termina cuando n / 2^i = 1, es decir, cuando $i = \log_2 n$.

Por lo tanto, la suma total es:

$$\Theta(1) + \Theta(2) + \Theta(4) + \cdots + \Theta(n)$$

4. Suma total y conclusión

La suma $1 + 2 + 4 + \cdots + n$ es geométrica.

Su valor es proporcional a $2n - 1$.

Entonces, el costo total satisface:

$$T(n) = \Theta(1 + 2 + 4 + \cdots + n) = \Theta(2n - 1)$$

Conclusión:
$$T(n) = \Theta(n)$$

Aunque la altura del árbol es $\Theta(\log n)$, el número total de nodos del árbol es lineal.

Por eso el algoritmo no es $O(\log n)$, sino $O(n)$.

5. Aplicación del Teorema Maestro

Recurrencia $T(n) = 2T(n/2) + \Theta(1)$

Se compara con la forma general: $T(n) = aT(n/b) + f(n)$

En este caso:

- $a = 2$
- $b = 2$
- $f(n) = \Theta(1)$

Calculamos: $n^{(\log_b a)} = n^{(\log_2 2)} = n$

Como $f(n) = \Theta(1) = O(n^{(1-\varepsilon)})$ para algún $\varepsilon > 0$, estamos en el caso 1 del Teorema Maestro.

6. Resultado final

Por el Teorema Maestro: $T(n) = \Theta(n)$

Interpretación:

- $T(n)$ es el tiempo de ejecución para un intervalo de n precios.
- $\Theta(1)$ es el costo fijo de combinar dos soluciones parciales.
- La recursión genera dos subproblemas de tamaño n/2 y luego combina en tiempo constante.

En consecuencia, el algoritmo analiza n precios en tiempo lineal.